Attributes reflection

Document #:	P3385R6 [Latest] [Status]
Date:	2025-09-26
Project:	Programming Language C++
Audience:	EWG, LEWG

Reply-to:	Aurelien Cassagnes <acassagnes@bloomberg.net>

Contents

§ 1 Revision history

R6

- Merge [P3678R0] into current paper following SG7 recommendation

— Address feedback from Sofia

— Remove `appertain`

— Add `has_attribute` metafunction

R5

— Remove in-place splicer syntax

— Add `appertain` metafunction

— Augment implementation feedback

§ 2 Introduction

Attributes are used to a great extent, and there is new attributes being added to the language somewhat regularly.
As reflection makes its way into our standard, we are missing a way for generic code to look into the attributes appertaining to an entity. That is what this proposal aims to tackle by introducing the building blocks.

§ 2.1 Motivating example

We expect a number of applications for attribute introspection to happen in the context of code generation [P2237R0], where for example, one may want to skip over `[[deprecated]]` members, explicitly tag python bindings with `@deprecated` decorators, etc.
The following example demonstrates cloning an aggregate while leaving out all deprecated members:

```
constexpr auto ctx = std::meta::access_context::current();

struct User {
  [[deprecated]] std::string name;
  [[deprecated]] std::string country;
  std::string uuidv5;
  std::string countryIsoCode;
};

template<class T>
struct MigratedT {
  struct impl;
  consteval {
    std::vector<std::meta::info> migratedMembers = {};
    for (auto member : nonstatic_data_members_of(^^T, ctx)) {
      if (!std::meta::has_attribute(member, ^^[[deprecated]])) {
        migratedMembers.push_back(data_member_spec(
          std::meta::type_of(member),
          {.name = std::meta::identifier_of(member)}
        ));
      }
```

```
    }
    define_aggregate(^^impl, migratedMembers);
  }
};

using MigratedUser =  MigratedT<User>::impl;
            static_assert(std::meta::nonstatic_data_members_of(^^User,
    ctx).size() == 4);
    static_assert(std::meta::nonstatic_data_members_of(^^MigratedUser,
    ctx).size() == 2);

int main() {
  MigratedUser newUser;
   // Uncomment the following line to show the deprecated fields are
    gone
  //   newUser.name = "bob";
  //
  // error: no member named 'name' in 'MigratedT<User>::impl'
  // 142 |   newUser.name = "bob";
  //
  newUser.uuidv5 = "bob";
}
```

[link].
A more fundamental motivation shows up when looking at `define_aggregate` design

```
namespace std::meta {
  struct data_member_options {
    struct name_type {
      template <typename T> requires constructible_from<u8string, T>
        consteval name_type(T &&);

      template <typename T> requires constructible_from<string, T>
        consteval name_type(T &&);
    };

    optional<name_type> name;
    optional<int> alignment;
    optional<int> bit_width;
    bool no_unique_address = false;
  };
}
```

Here we have 2 attributes showing up in `alignas` and `[[no_unique_adress]]`. We have no way to appertain `[[deprecated]]`, `[[maybe_unused]]`, or any other attributes.
If one wants to explicitly tell their compiler to enforce this attribute, they may want to tag `[[msvc::no_unique_address]]` out of caution.
Having a vehicle to solve this design concern is a major motivation for this paper.

§ 3 Scope

Before longer discussions, we can give a quick rundown of what the initial support is expected to be

Category	Sample	Rationale
🟢 Standard	`[[nodiscard]]`	
🟢 Vendor variant	`[[clang::warn_unused_result]]`	
🟢 Vendor specific	`[[gnu::constructor]]`	
🟠 Vendor specific	`[[clang::availability]]`	Complexity
🔴 Unknown	`[[my::attribute(freeform arg)]]`	Undefined parsing

Complexity here refers to our implementation experience dealing with positional arguments. Experienced implementers may feel differently, what remains true is that we want to leave the choice to implementers to opt out for problematic attributes.

§ 3.1 Argument clause

Let us recap here what are the attributes 9.13 [dcl.attr] found in the standard and their argument clause

Attribute	Argument-clause
assume	conditional-expression
deprecated	unevaluated-string
fallthrough	N/A
indeterminate	N/A
likely	N/A
maybe_unused	N/A
nodiscard	unevaluated-string
noreturn	N/A
no_unique_address	N/A
unlikely	N/A

Revisions of this proposal up to [P3385R2] were willfully ignoring *attribute-argument-clause* when it comes to comparison. Feedback post Wroclaw was unanimous on treating the argument clause as a salient property (and so starting from the second revision `^^[[nodiscard("foo")]] != ^^[[nodiscard("bar")]]`).
Feedback from implementers has been that while it brings no concern for attributes like `nodiscard`, it is more an open question [1] when it comes to an attribute like `assume` accepting an expression as argument… Should `^^[[assume(i + 1)]]` compare equal to `^^[[assume(1 + i)]]`) ?
Ultimately we leave to implementations the freedom to define the comparisons of the argument clause for `assume`. To keep things simple our experimental implementation does not transform the expression into a canonical representation before evaluating equality via profiling and so `^^[[assume(i + 1)]]` is not the same as `^^[[assume(1 + i)]]`).

§ 3.2 Support, optionality and self consistency

There is a long standing and confusing discussion around the ignorability of attributes. We'll refer the reader to [P2552R3] for an at-length discussion of this problem, and especially with regard to what 'ignorability' really means. Another interesting conversation takes place in [P3254R0], around the `[[no_unique_address]]` case, and around `[[assume]]` in CWG 3038, all of which serves again to illustrate that the tension around so called ignorability should not be considered a novel feature of this proposal.

Here we introduce an alternative terminology to guide the conversation in the context of reflection and attributes

— Unknown attributes are **unsupported** (e.g, `[[my::attribute]]`).

— Implementation specific attributes problematic to reflect are **unsupported** (e.g., `[[clang::availability(macos,introduced=10.4,deprecated=10.6,obsoleted=10.7)]]`)

— Every other attributes (incl. standard) are **supported** (e.g., `[[nodiscard]]`, `[[gnu::constructor(100)]]`)

With this category in place, we make the following design choice

1. Creating reflection of unsupported attribute is ill-formed (diagnostic required)
2. `attributes_of` does not return unsupported attributes

This is done to allow implementers plenty room to grow the set of supported attributes w/o worrying about breaking code. The current practice around creating reflection of problematic constructs (such as `using-declarator`) is to be ill-formed and we'll follow that here. Diagnostic in those cicrumstances are not hard to emit, and so we should do so.
The other alternative would be to yield a null reflection and emit a warning, which we think is a worse option.

§ 4 Proposed Features

§ 4.1 Reflection expression

Our proposal advocates to support reflect expression like

```
constexpr auto r = ^^[[nodiscard("keepMe")]];
```

The result is a reflection value embedding salient property of the attribute which are the attribute namespace, token and the argument clause if any.

§ 4.2 Metafunctions

We propose to add a couple of metafunctions to what is available in <meta>. In addition, we will extend support to attributes in the other metafunctions when it makes sense.

§ 4.2.1 attributes_of

```
namespace std::meta {
  consteval auto attributes_of(info construct) -> vector<info>;
}
```

`attributes_of()` returns a vector of reflections representing all individual attributes that appertain to `construct`.
Simple example follows

```
enum class [[nodiscard("Error discarded")]] ErrorCode {
  Disconnected,
  ConfigurationIncorrect,
  OutdatedCredentials,
};

  static_assert(attributes_of(^^ErrorCode)[0]  ==  ^^[[nodiscard("Error
    discarded")]]);
```

In the case where an entity is legally redeclared with different attribute arguments, `attribute_of` return one of those.

```
enum class ErrorCode;
enum class [[nodiscard("Error discarded")]] ErrorCode;
enum class [[nodiscard]] ErrorCode {
  Disconnected,
  ConfigurationIncorrect,
  OutdatedCredentials,
};

// Either of [[nodiscard("Error discarded")]] or [[nodiscard]]
static_assert(attributes_of(^^ErrorCode).size() == 1);
```

§ 4.2.2 has_attribute

```
namespace std::meta {
  enum class attribute_comparison {
    ignore_namespace, // Namespace is ignored during the comparison
    ignore_argument,  // Arguments are ignored during the comparison
  };

  consteval auto has_attribute(info                 construct,
                               info                      attribute) ->
    bool;
```

```
  consteval auto has_attribute(info                 construct,
                               info                 attribute,
                               attribute_comparison policy) -> bool;
}
```

`has_attribute()` returns true if the specified `attribute` is found appertaining to `construct`, false otherwise.
Simple example follows

```
struct [[clang::consumable(unconsumed)]] F {
    [[clang::callable_when(unconsumed)]] void f() {}
};

                          static_assert(std::meta::has_attribute(^^F::f,
    ^^[[clang::callable_when(unconsumed)]]));
```

[link]

The overload with `policy` parameter allows a combo of flags to dictate what part of an attribute are meaningful to the comparison. This comes in handy when we want to find out an attribute, ignoring the vendor prefix and or the particular message that is being attached here and there.

```
[[gnu::deprecated("Standard deprecated")]] void f() { }

// Ignore both the namespace and the argument
static_assert(std::meta::has_attribute(
  ^^f,
  ^^[[deprecated]],
    std::meta::attribute_comparison::ignore_namespace | std::meta::at-
     tribute_comparison::ignore_argument
));
```

[link]

§ 4.2.3 is_attribute

```
namespace std::meta {
  consteval auto is_attribute(info r) -> bool;
}
```

`is_attribute()` returns true if `r` represents an attribute, it returns false otherwise. Its use is trivial

```
static_assert(is_attribute(^^[[nodiscard]]));
```

§ 4.2.4 identifier_of, display_string_of

Given a reflection `r` designating an attribute, `identifier_of(r)` (resp. `u8identifier_of(r)`) should return a `string_view` (resp. `u8string_view`) corresponding to the `attribute-token`.

A sample follows

```
                    static_assert(identifier_of(^^[[clang::warn_unused_re-
        sult("message")]] == "clang::warn_unused_result")); // true
              static_assert(identifier_of(^^[[nodiscard("message")]]    ==
        "nodiscard")); // true
```

Given a reflection `r` that designates an individual attribute, `display_string_of(r)` (resp. `u8display_string_of(r)`) returns an unspecified non-empty `string_view` (resp. `u8string_view`). Implementations are encouraged to produce text that is helpful in identifying the reflected attribute for display purpose. In the preceding example we could imagine printing `[[clang::warn_unused_result("message")]]`as it might be better fitted for diagnostics.

§ 4.2.5 data_member_spec, define_aggregate

To support arbitrary attributes appertaining to data members, we'll need to augment `data_member_options` to encode attributes we want to attach here.

The structure changes thusly:

```
    namespace std::meta {
      struct data_member_options {
        struct name_type {
          template <typename T> requires constructible_from<u8string, T>
            consteval name_type(T &&);

          template <typename T> requires constructible_from<string, T>
            consteval name_type(T &&);
        };

        optional<name_type> name;
        optional<int> alignment;
        optional<int> bit_width;
-       bool no_unique_address = false;
+       [[deprecated]] bool no_unique_address = false;
+       vector<info> attributes;
      };
    }
```

From there building an aggregate piecewise proceeds as usual

```
struct Empty{};
struct [[nodiscard]] S;
consteval {
  define_aggregate(^^S, {
    data_member_spec(^^int, {.name = "i"}),
    data_member_spec(^^Empty, {.name = "e",
                               .attributes = {^^[[msvc::no_unique_ad-
    dress]]}})
  });
}

// Equivalent to
// struct [[nodiscard]] S {
//   int i;
//   [[msvc::no_unique_address]] struct Empty { } e;
// };
```

Passing attributes through the above proposed approach is well in line with the philosophy of leveraging `info` as the opaque vehicle to carry every and all reflections.

§ 5 Proposed wording

§ 5.1 Language

§ 6.9.2 [basic.fundamental] Fundamental types

Augment the description of `std::meta::info` found in new paragraph 17 to add attribute as a valid representation to the current enumerated list

17 A value of type `std::meta::info` is called a reflection. There exists a unique *`null reflection`*; every other reflection is a representation of

…

17-20 - a direct base class relationship ([class.derived.general])~~, or~~,

17-21 - a data member description ([class.mem.general])~~.~~, or

17-22 - an attribute([dcl.attr])

Update 17.18 `Recommended practices` to remove attributes from the list

17-2 *`Recommended practice`*: Implementations should not represent other constructs specified in this document, such as *`using-declarators`*, partial template specializations, ~~attributes,~~ placeholder types, statements, or expressions, as values of type std::meta::info.

§ 7.6.2.10 [expr.reflect] The reflection operator

Edit *`reflect-expression`* production rule to support reflecting over attributes

reflect-expression:
`^^` `::`
`^^` *reflection-name*
`^^` *type-id*
`^^` *id-expression*
`^^` `[[` *attribute* `]]`

Add the following paragraph after the last paragraph of 7.6.2.10 [expr.reflect] to describe the new rule `^^` `[[` *attribute* `]]`

(11.1) A *`reflect-expression`* of the form `^^[[` *`attribute`* `]]` for attribute described in this document [dcl.attr], represents said attribute.

(11.2) For an *attribute* `r` non described in this document, computing the reflection of `r` is ill-formed absent implementation-defined guarantees with respect to said *attribute* .

§ 7.6.10 [expr.eq] Equality Operators

Update 7.6.10 [expr.eq]/6 to add a clause for comparing reflection of attributes

…
(6.7) - represent equal data member descriptions([class.mem.general]),
(6.8) - represent identical attribute ([dcl.attr])
[*Example:*

```
static_assert(^^[[nodiscard]] == ^^[[nodiscard]]);
           static_assert(^^[[nodiscard("keep")]]       ==
      ^^[[nodiscard("keep")]]);
   static_assert(^^[[nodiscard]]   !=   ^^[[deprecated]]);
      // different attribute token
           static_assert(^^[[nodiscard("keep")]]       !=
      ^^[[nodiscard("keep too")]]); // different argument
      clause
static_assert(^^[[nodiscard("keep")]] != ^^[[nodiscard]]);
      // different argument clause
```

— end example]
and they compare unequal otherwise.

§ 9.13.1 [dcl.attr.grammar] Attribute syntax and semantics

Add a new paragraph to describe when are two standard attribute considered identical. Two `[[assume]]` attributes are always identical, for others we compare the attribute to-

kens which must match, and their clause for simple clause.

9+ For any two attributes r_1 and r_2 described in this document, r_1 and r_2 are identical if their *attribute-token* are identical and
- *attribute-token* is `assume`, or
- r_1 and r_2 admit no *attribute-argument-clause*, or
- r_1 and r_2 admit optional *attribute-argument-clause* and they are both empty or
- r_1 and r_2 admit *attribute-argument-clause* of the form `( unevaluated-string )` and r_1 and r_2 *balanced-token-seqs* are identical.
Otherwise r_1 and r_2 are not identical.

(Note: Identity between attributes not described in this document is implementation defined)
[*Example:*

```
[[nodiscard("A")]][[deprecated]] void f() {}
[[nodiscard("A")]][[deprecated("B")]] void g() {} // the
      'deprecated' attributes are not identical
                                                    // the
      'nodiscard' attributes in f() and g() are identical

static_assert(^^[[gnu::constructor(2)]] == ^^[[gnu::con-
      structor(1 + 1)]]); // implementation defined
```

— *end example*]

§ 13.10.3.1 [temp.deduct.general] Template argument deduction

Add a paragraph at the bottom to indicate that a dependent expression in an attribute does participate in SFINAE when that dependent expression is ill-formed for a given instantiation.

8+ Substitution into an *attribute* is in the immediate context.
[*Example:*

```
template <class T>
auto foo(T) -> decltype(^^[[assume(T::value == 0)]]);

void foo(...);

foo(1);  // call the second overload
```

— *end example*]

§ 5.2 Library

§ 21.4.1 [meta.syn] Header <meta> synopsis

Add to the [meta.reflection.queries] section from the synopsis, the metafunctions `is_attribute`, `attributes_of` and `has_attribute` along with the `attribute_comparison` enumeration.

```
...
 // [meta.reflection.queries], reflection queries
...


consteval bool is_attribute(info r);

consteval vector<info> attributes_of(info r);

enum class attribute_comparison {
    ignore_namespace,
    ignore_argument,
};

consteval bool has_attribute(info r, info a);

 consteval bool has_attribute(info r, info a, attribute_-
      comparison flags);
```

§ 21.4.6 [meta.reflection.names] Reflection names and locations

Introduce a subclause to `has_identifier` describing the return value to be `true` for attribute reflection

```
consteval bool has_identifier(info r);
```

1 *Returns*:

…

(1.14) — Otherwise, if r represents an attribute, then `true`

Introduce a subclause to `identifier_of`, `u8identifier_of`, describing the return value of attribute reflection to be the `attribute-token`. Renumber the last clause appropriately.

```
consteval string_view identifier_of(info r);
consteval string_view u8identifier_of(info r);
```

3 *Returns*: An NTMBS, encoded with E, determined as follows:

…

(3.6-) — Otherwise, if `r` represents an attribute `a`, then the *`attribute-token`* of `a`

§ 21.4.7 [meta.reflection.queries] Reflection queries

Add the new clauses to support new metafunctions, and the new enumeration.

* `consteval bool is_attribute(info r);`

Returns: `true` if `r` represents an attribute. Otherwise, `false`.

* `consteval vector<info> attributes_of(info r);`

Returns: A vector `v` containing reflections of all attributes appertaining to the entity represented by `r`, such that `is_attribute(`v_i`)` is true for every attribute v_i in `v`. The ordering of `v` is unspecified.
[*Example:*

```
enum class [[nodiscard, deprecated]] Result { Success,
    Failure };

static_assert(attributes_of(^^Result).size() == 2);
```

— *end example*]

Add a new table to describe the comparison policy `attribute_comparison` between attributes. Add a new clause to describe `has_attribute`

```
enum class attribute_comparison {
    ignore_namespace = unspecified,
    ignore_argument = unspecified,
};
```

* The type `attribute_comparison` is an implementation-defined bitmask type ([bitmask.types]). Setting its elements has the effect listed in Table (∗) [tab:meta.reflection.queries]

`attribute_comparison` effects [tab:meta.reflection.queries]

Element	Effect(s) if set
`ignore_namespace`	Specifies that the *`attribute-namespace`* is ignored when comparing attributes

Element	Effect(s) if set
`ignore_argument`	Specifies that the *`attribute-argument-clause`* is ignored when comparing attributes

* `consteval bool has_attribute(info r, info a);`

Returns: True if `a` was found appertaining to the construct `r`.
Throws: `meta::exception` unless `is_attribute(a)` is `true`.

* `consteval bool has_attribute(info r, info a, attribute_comparison flags);`

Returns: True if `a` was found appertaining to the construct `r`. The bitmasks specified in `flags` determine which components of an attribute are considered significant for matching purpose.
Throws: `meta::exception` unless `is_attribute(a)` is `true`.

§ 21.4.16 [meta.reflection.define.aggregate] Reflection class definition generation

— Change `data_member_options` definition to mark the current `no_unique_address` member deprecated and add the `attributes` data member.

— Update the *`Returns:`* description to accomodate for the new `attributes` data member.

— Update the *`Effects:`* to describe the new `attributes` data member effects.

```
  namespace std::meta {
    struct data_member_options {
      struct name_type {
                       template   <typename   T>   requires
        constructible_from<u8string, T>
          consteval name_type(T &&);

                      template   <typename   T>   requires
        constructible_from<string, T>
          consteval name_type(T &&);
      };

      optional<name_type> name;
      optional<int> alignment;
      optional<int> bit_width;

-       bool no_unique_address = false;
+       [[deprecated]] bool no_unique_address = false;
+       vector<info> attributes;
```

```
    };
  }

...

Returns: A reflection of a data member description (T, N, A,
        W, NUA, AT) (11.4.1 [class.mem.general]) where

  (5.1) - T is the type represented by delias(type),

  (5.2) - N is either the identifier encoded by options.name
        or ⊥ if options.name does
          not contain a value,

  (5.3) - A is either the alignment value held by option-
        s.alignment or ⊥ if options.alignment does not
          contain a value,

  (5.4) - W is either the value held by options.bit_width or
        ⊥ if options.bit_width does not contain a
          value, and

        (5.5) - NUA is the value held by
        options.no_unique_address.

+ (5.6) - AT is the value held by options.attributes.


...

Effects: Produces an injected declaration D ([expr.const])
        that provides a definition for C with properties
  as follows:

...

  If oK.name does not contain a value, it is an unnamed bit-
        field. Otherwise, it is a non-static data member
   with an identifier determined by the character sequence
        encoded by u8identifier_of(rK) in UTF-8.


+ Every reflection r in ok.attributes is appertained to D if
        is_attribute(r) is true

...
```

§ 5.3 Feature-test macro

The attribute reflection feature is guarded behind macro, augment 15.12 [cpp.predefined]

```
__cpp_impl_reflection 2025XXL
__cpp_impl_reflection_attributes 2025XXL
```

§ 6 Feedback

§ 6.1 Poll

§ 6.1.1 P3385R1: SG7, Nov 2024, WG21 meetings in Wroclaw

— SG7 encourages more work on reflection of attributes as described in the paper: No objection to unanimous consent

§ 6.1.2 P3385R2: SG7, Dec 2024, Telecon

— SG7 wants to support namespaced attributes: No objection to unanimous consent.

— SG7 wants to support "easy" arguments of attributes: No objection to unanimous consent.

— SG7 wants to support arguments (full expressions) of attributes: Not consensus.

— SG7 considers the paper high-priority and forwards it to LEWG and EWG for C++26: Not consensus.

— SG7 forwards this paper to LEWG and EWG for C++29: Not consensus.

§ 6.1.3 P3385R3: SG7, Feb 2025, Hagenberg

— SG7 wants to support token source type arguments: Not consensus

— SG7 wants to forward to EWG and LEWG as is: Consensus

§ 6.1.4 D3385R6: SG7/EWG, June 2025, Sofia

— SG7 wants to allow arbitrary attributes support via define_aggregate: Consensus (recommendation to merge into P3385)

— EWG would prefer to see this paper without the ability to 'appertain' an attribute to an entity : Consensus

— EWG encourages more work in the direction of the paper that better exposes the details of the attributes from a querying perspective: Consensus

— EWG encourages more work on reflecting attributes in the direction of the paper: Not consensus

§ 6.2 Implementation

The features presented here are available on compiler explorer[2].

§ 7 References

[P2237R0] Andrew Sutton. 2020-10-15. Metaprogramming.
https://wg21.link/p2237r0

[P2552R3] Timur Doumler. 2023-06-14. On the ignorability of standard attributes.
https://wg21.link/p2552r3
[P3254R0] Brian Bi. 2024-05-22. Reserve identifiers preceded by @ for non-ignorable annotation tokens.
https://wg21.link/p3254r0
[P3385R2] Aurelien Cassagnes, Roman Khoroshikh, Anders Johansson. 2024-12-12. Attributes reflection.
https://wg21.link/p3385r2
[P3678R0] Aurelien Cassagnes. 2025-05-15. Arbitrary attributes in define_aggregate.
https://wg21.link/p3678r0

---

1. It is mostly an academic question since `[[assume]]` can only appertain to the null statement, no calls to `attributes_of` could return such a reflection. The only way to get one is to construct one explicitly via `constexpr auto r = ^^[[assume(expr)]];` and the utility of doing so is null.↩

2. Compiler explorer↩